Model analysis: baseline evaluation and image-size tuning

Baseline evaluation and `imgsz` tuning by Ben Nichiporik · November 2026

This part of the report evaluates the baseline gesture detector in detail (Section 1), compares it with three models fine-tuned at different image sizes (Section 2), and describes the problems we ran into in these steps and how we handled them (Section 3). The detector finds hands in a photo and classifies each one as one of four gestures: **palm**, **fist**, **like** (thumbs-up) or **dislike** (thumbs-down).

1. Baseline model evaluation

1.1 Model and training

The baseline is **YOLOv8s** (11.1 M parameters) pretrained on COCO and fine-tuned on our data with all layers trainable (transfer learning). It was trained for 50 epochs, and the checkpoint with the best validation fitness (epoch 41) is the baseline model (`models/baseline_best.pt`). Table 1 lists the hyperparameters. They are defined once in `gesture_eval.py` (`BASELINE`), so each member's tuning runs change exactly one of them. Table 2 shows how the data was split.

Table 1. Baseline hyperparameters.

setting	value
model	YOLOv8s, COCO-pretrained, all layers fine-tuned
image size (imgsz)	640
epochs	50 (early-stopping patience 15)
batch	16
optimizer	SGD, lr0 0.01, momentum 0.937, weight decay 0.0005
augmentation	flip left-right 0.5, up-down 0.0; Ultralytics defaults (mosaic, HSV, scale, translate)
seed	0, deterministic

Table 2. Images per split and class. HaGRID images 001-368 of each class are used for training, validation and part of the test set. Our own photos (369-400, 8 per member per class) are used only for testing.

split · source	palm	fist	like	dislike	total
train · HaGRID	320	320	320	320	1280
val · HaGRID	40	40	40	40	160
test · HaGRID	8	8	8	8	32
test · our photos	32	32	32	32	128

1.2 Evaluation protocol

Every model in the project is scored by the same code (`gesture_eval.py`) on three sets: HaGRID validation (used to choose the checkpoint), the full test split, and **our own photos only** (128 images, 154 hands). The last one is the honest estimate of how the photo booth will perform: those photos were never seen in training and look like real use, with different people, places and lighting. We report the standard Ultralytics metrics (precision, recall, mAP50 and mAP50-95). We also score our photos at a fixed **deployment threshold**: a prediction counts if its confidence is at least 0.25, and it is correct if its box overlaps a labelled hand with IoU ≥ 0.5 and the class matches. Because the test set is small, we resample the photos (bootstrap, 2,000 resamples) to put 95% confidence intervals on precision, recall and F1.

1.3 Training behaviour

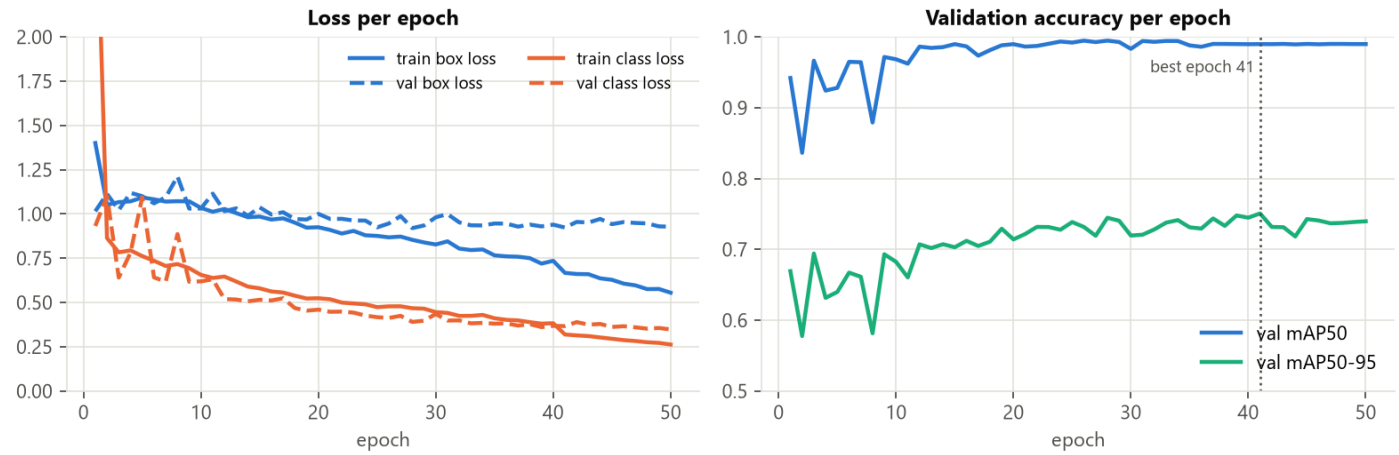


Figure 1. Baseline training curves. Left: box and class loss on the training and validation sets. Right: validation mAP50 and mAP50-95. The dotted line marks the selected checkpoint.

Transfer learning works quickly. Validation mAP50 is already 0.94 after one epoch, because the COCO-pretrained backbone already provides general shape and edge features. The best epoch (41) reaches mAP50 0.990 and mAP50-95 0.751. The validation losses keep falling alongside the training losses through epoch 50, and early stopping never triggered, so the model is not overfitting *to HaGRID*. The overfitting that matters is to HaGRID's *style* of photo, which only the test on our own photos reveals.

1.4 Overall results

Table 3. Baseline results on each evaluation set (Ultralytics metrics).

evaluation set	precision	recall	F1	mAP50	mAP50-95
validation (160)	0.990	0.984	0.987	0.990	0.748
test, all (160)	0.911	0.811	0.858	0.875	0.469
test, our photos only (128)	0.894	0.775	0.830	0.845	0.418

The validation scores are inflated. HaGRID validation images come from the same collection as the training images: one person standing and facing a phone camera, mostly indoors, with one hand per photo. On our own photos, mAP50 drops by 0.15 (to 0.845) and mAP50-95 by 0.33. The larger drop in mAP50-95 means the model usually *finds* the hand, but its box doesn't line up tightly with ours: correct detections have a mean IoU of only 0.76 with our labels. Part of this comes from annotation style rather than model error, because each member drew boxes in MakeSense slightly differently (some include the wrist). At conf 0.25 on our photos, the 95% bootstrap intervals are precision 0.77-0.90, recall 0.74-0.87 and F1 0.76-0.88. A single model's F1 is therefore uncertain by about **±0.06**, which is the bar a tuned model has to clear (Section 2).

1.5 Per-class performance

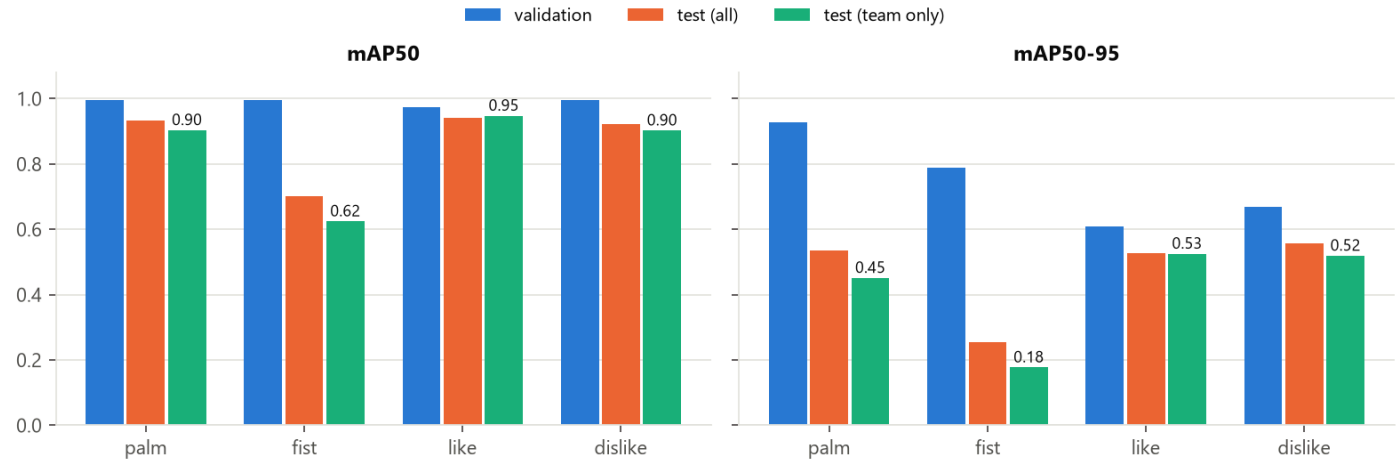


Figure 2. mAP50 and mAP50-95 per class on validation, the full test split and our photos only (labelled values).

Table 4. Per-class results on our photos. Precision, recall and mAP from Ultralytics; missed and wrong-class counts at conf 0.25, IoU 0.5.

class	precision	recall	mAP50	mAP50-95	hands	missed	wrong class
palm	0.927	0.765	0.904	0.452	33	5	3
fist	0.758	0.611	0.625	0.177	36	12	1
like	0.927	0.909	0.948	0.526	42	2	1
dislike	0.963	0.814	0.903	0.519	43	5	1

Palm and like transfer well because their shapes are distinctive (spread fingers, upward thumb). **Fist is the weakest class**: 12 of 36 fists are missed entirely, 3 of them only because their labels were drawn too loosely (Section 1.9). A fist is a compact, low-texture blob that looks like many other things at small scale, and several of our fists are raised in crowd scenes with busy backgrounds, which HaGRID never shows. **Dislike is confused with fist** in both directions: a thumbs-down seen from the side looks like a fist with a short thumb, and these are the highest-confidence errors.

1.6 Errors at the deployment threshold

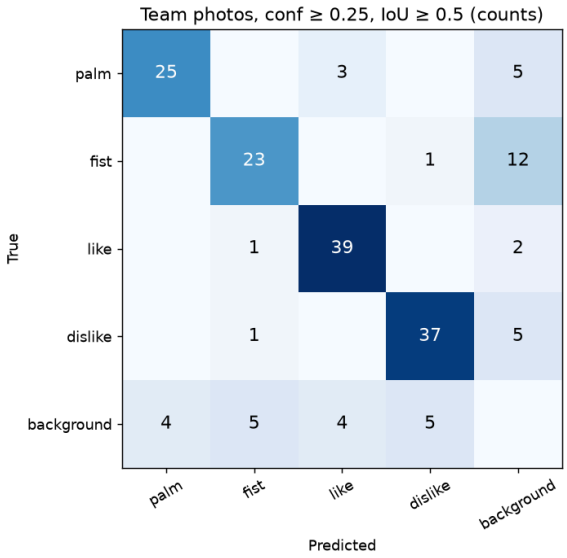


Figure 3. Confusion matrix on our photos at conf 0.25, IoU 0.5 (counts).

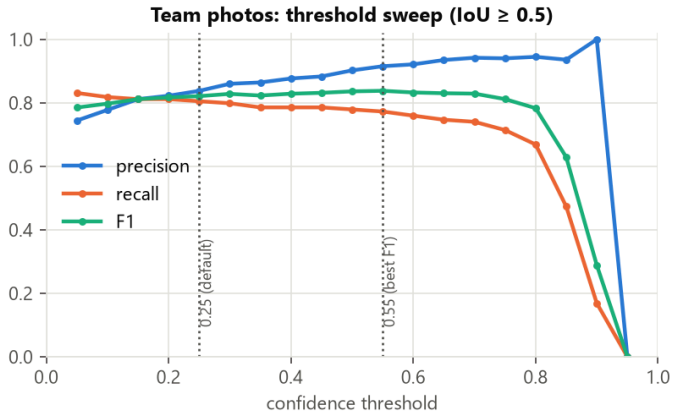


Figure 4. Precision, recall and F1 on our photos as the confidence threshold changes.

At conf 0.25 the model makes 148 predictions for 154 labelled hands in our 128 photos: **124 correct**, **24 hands missed**, **6 found with the wrong class** and **18 predictions on background** (no labelled hand there; by predicted class: dislike 5, fist 5, palm 4, like 4). Missing a hand is the most common error. F1 is highest at conf 0.55 (precision 0.92, recall 0.77, F1 0.84), compared with precision 0.84 and recall 0.81 at 0.25. For a photo booth, a false trigger (taking a picture nobody asked for) is worse than a missed gesture (the user just holds it a moment longer). We therefore recommend a threshold of about 0.55, plus requiring the gesture over several consecutive frames.

1.7 Where the model fails

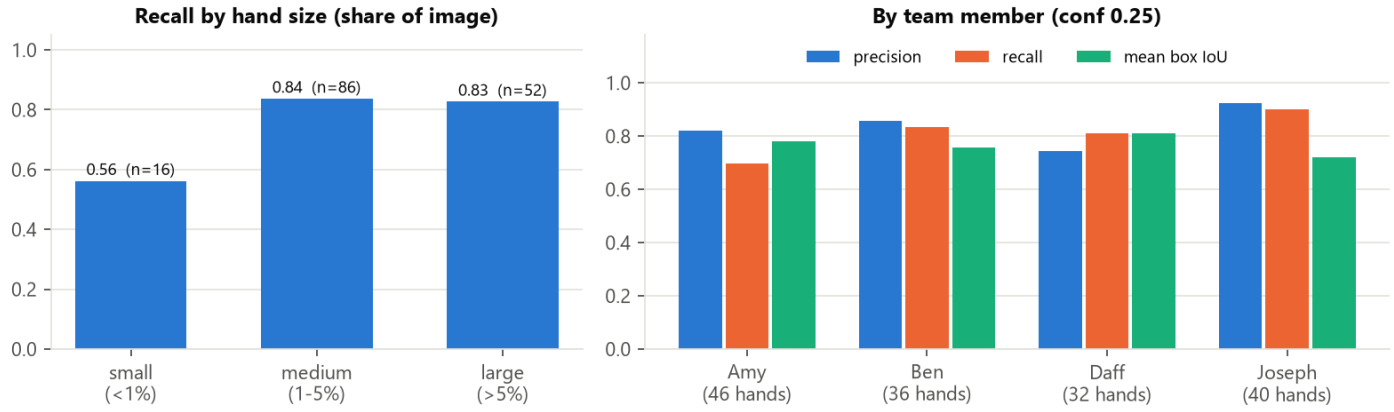


Figure 5. Left: recall on our photos by hand size (share of the image). Right: precision, recall and mean box IoU for each member's photos (conf 0.25).

Table 5. Recall by hand size on our photos.

hand size	hands	recall	missed
small (<1%)	16	0.56	6
medium (1-5%)	86	0.84	13
large (>5%)	52	0.83	5

Table 6. Results by the member who took the photos.

member	hands	predictions	correct	precision	recall	mean IoU
Amy	46	39	32	0.82	0.70	0.78
Ben	36	35	30	0.86	0.83	0.76
Daff	32	35	26	0.74	0.81	0.81
Joseph	40	39	36	0.92	0.90	0.72

Hand size is the clearest driver. The median labelled hand covers 3.5% of the image, but 16 hands cover under 1%, and only 56% of those are found. At imgsz 640 those hands are only about 25-62 px across after resizing, which leaves little detail. Size alone is not the cause, though: 386 of the 1,280 training hands (30%) are also under 1% of the image, and the median training hand (1.6%) is smaller than ours. The difference is **context**. HaGRID's small hands belong to one person posing in a plain room, while ours are in crowds, outdoors and at odd angles. **Per member**, the differences follow photo choice and labelling style. Amy's photos hold 10 of the 16 small hands, which lowers recall there. Joseph's photos have the lowest mean box IoU (0.72) even though their precision is 0.92: the model finds those hands, but the boxes are drawn differently.

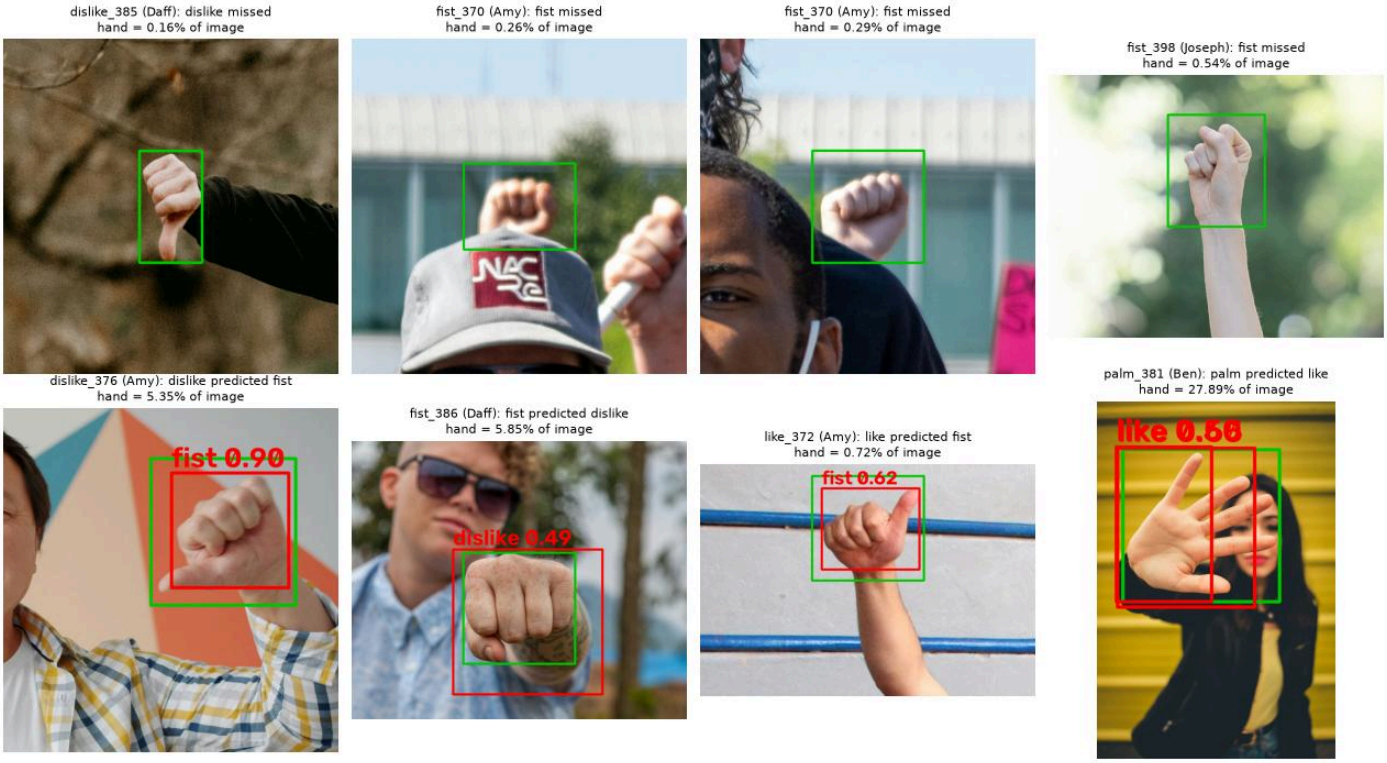


Figure 6. Failure cases. Green = our label, red = prediction (conf ≥ 0.25). Top row: the smallest missed hands. Bottom row: wrong-class predictions (fist/dislike and palm/like confusions).

1.8 Speed

Per image at imgsz 640 (batch 1, NVIDIA RTX 5060 Ti): preprocess 2.1 ms, inference 12.1 ms and postprocess 2.4 ms, **16.6 ms in total (about 60 frames per second)**. The model file is 23 MB (28.4 GFLOPs). A webcam photo booth needs about 15-30 frames per second, so a GPU has plenty of headroom. CPU-only speed was not measured.

1.9 Limits of this evaluation

- **Small test set.** 154 hands in 128 photos; the F1 interval in Section 1.4 is 0.12 wide.
- **Duplicate photo.** `dislike_388` (Daff) and `dislike_393` (Joseph) are the same picture, so those hands count twice and the test set has 127 distinct photos.
- **Labelling errors (human error).** Some test labels are wrong and were left as they are: boxes drawn much larger than the fist (`fist_380`, `fist_381`, `fist_384`), gesturing hands that were never labelled (`fist_376`, 2 raised fists; `fist_382`, 2 raised fists in the crowd; `palm_388`, palms of the people behind the main subject) and an unclear gesture (`palm_376`, hand held across the face, back of the hand to the camera). A correct, tight box on a loose label overlaps it by less than 0.5 IoU, so it counts as a miss plus a false positive. These errors cause 3 of the 24 missed hands and 4 of the 18 background predictions, so the scores on our photos are slightly pessimistic.
- **Optimistic validation.** Validation is HaGRID-only, so choosing the checkpoint by it does not reward robustness to our kind of photo.
- **One training seed.** Run-to-run variation was not measured beyond the bootstrap over test photos.

---

2. Fine-tuned models: image size (`imgsz`), Ben Nichiporik

2.1 Hypothesis

`imgsz` is the size (longest side, in pixels) that every image is resized to for training and inference. Section 1.7 showed that the baseline's main weakness is small hands, which are only about 25-62 px across at imgsz 640 (the median hand

is 119 px). **A larger imgsz (800, 960) should keep more pixels per hand and raise recall on small hands**, especially fists, at the cost of slower training and inference. A smaller imgsz (480) shows how much accuracy we would lose for speed, which matters for a real-time photo booth.

2.2 Method

I trained three models that are identical to the baseline except for `imgsz` (480, 800 and 960): same data split, seed, epochs, early stopping and augmentation. Each model was scored with the same code as the baseline, at the size it was trained at, and the headline is again our own photos (128 images, 154 hands). Validation is saturated for every size (mAP50 0.990-0.995), so it can't separate the runs. I also compared the runs hand by hand: which of the baseline's errors each run fixed, and which of its correct detections it broke.

Table 7. Training runs. *8.5 min on an idle GPU (15.3 min in a re-run on a shared GPU, with an identical model). Inference is per image at batch 16 and varies by 1-2 ms between runs.

run	imgsz	epochs run	best epoch	train min	ms / image
imgsz_480	480	50	41	5.3	2.9
baseline	640	50	41	8.5*	4.8
imgsz_800	800	50	38	17.0	8.8
imgsz_960	960	41	26	26.0	12.4

2.3 Results

Table 8. Fine-tuned models compared with the baseline on our photos. Δ = change from the baseline. Small hands (under 1% of the image) found and fixed / broken hands at conf 0.25, IoU 0.5.

imgsz	val mAP50	mAP50	mAP50-95	fist mAP50	precision	recall	F1 @ 0.25	Δ mAP50	Δ F1	small hands	fixed / broken
480	0.995	0.830	0.426	0.618	0.855	0.780	0.813	−0.015	−0.008	8 / 16	9 / 11
640 (baseline)	0.990	0.845	0.418	0.625	0.894	0.775	0.821	—	—	9 / 16	—
800	0.994	0.848	0.429	0.642	0.894	0.765	0.808	+0.003	−0.013	10 / 16	9 / 9
960	0.995	0.828	0.426	0.594	0.880	0.751	0.785	−0.017	−0.036	9 / 16	12 / 19

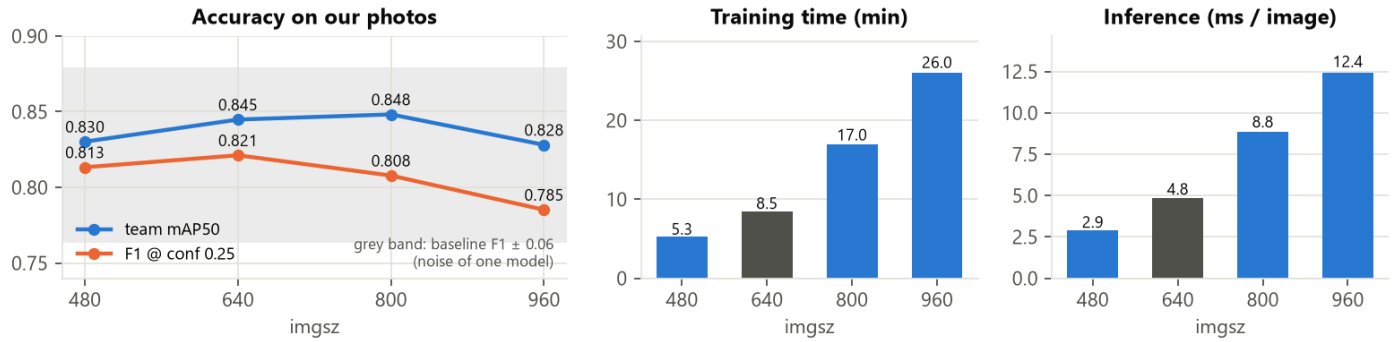


Figure 7. Left: mAP50 and F1 on our photos for each image size; the grey band is the baseline F1 ± 0.06, the uncertainty of one model. Middle and right: training time and inference time per image.

No image size beats the baseline by more than noise. Team mAP50 moves by at most 0.017, far inside the ±0.06 uncertainty from Section 1.4.

- **imgsz 800 is the best on paper, but only just.** It has the highest mAP50 (0.848, +0.003), mAP50-95 (0.429) and fist mAP50 (0.642, +0.018). Its precision equals the baseline's and its recall is lower (0.765 vs 0.775), so F1 at the deployment threshold is slightly *worse* (0.808 vs 0.821). It fixed 9 of the baseline's errors and broke 9 of its correct detections.
- **imgsz 960 is the worst for its cost.** It early-stopped at epoch 41 (best epoch 26), took 3× the baseline's training time, fixed 12 hands but broke 19, and its recall on large hands fell from 0.83 to 0.71.
- **imgsz 480** is the fastest and only slightly worse (−0.015 mAP50), but it finds the fewest small hands (8 / 16).

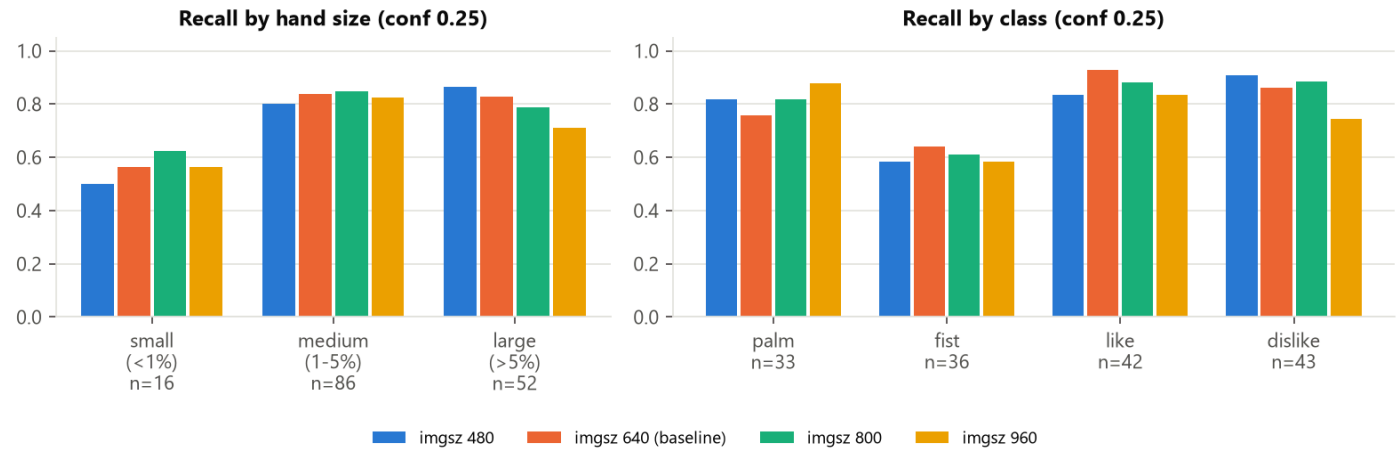


Figure 8. Recall on our photos at conf 0.25 by hand size (left) and by class (right) for each image size.

Small hands barely moved: 8/16 at 480, 9/16 at 640, 10/16 at 800, 9/16 at 960. One extra hand at 800 is not a real improvement on a sample this small, so a larger image size did not fix the baseline's main weakness. Fist stays the weakest class at every size (recall 0.58, 0.64, 0.61, 0.58).

2.4 Why a larger image size didn't help

Resolution was not the real problem. As Section 1.7 showed, the training set already has plenty of small hands (30% are under 1% of the image), and its median hand is smaller than ours. The gap is **scene context**: our missed hands are in crowds, outdoors, at odd angles and against busy backgrounds. A larger input gives the model more pixels, but it doesn't teach it what those scenes look like.

imgsz 960 may be under-trained. Its best checkpoint is from epoch 26, so it effectively trained for about half as long as the others. That probably explains part of its drop on large hands.

Table 9. Control: mAP50 on our photos of the baseline weights (trained at 640) run at each size, compared with a model trained at that size.

imgsz	baseline weights run at this size	model trained at this size
480	0.837	0.830
640 (baseline)	0.845	0.845
800	0.823	0.848
960	0.785	0.828

The control supports this. Running the 640-trained baseline at a larger size *lowers* mAP50 (0.845 → 0.823 at 800 → 0.785 at 960). Our hands are already larger on average than HaGRID's, so upscaling pushes them further from the sizes seen in training. Training at the larger size only wins back what that mismatch costs.

2.5 Conclusion

The hypothesis was not supported. **My best value is imgsz 800** (best mAP50, mAP50-95 and fist mAP50), so it is my entry in the group comparison. However, its gain over the baseline (+0.003 mAP50) is effectively a tie, and it costs about 2× the training time and 1.8× the inference time. For the photo booth itself, **imgsz 640 remains the better choice**: it has the best F1 at the deployment threshold (0.821) and the best balance of speed and accuracy. The real weakness should be addressed through the data: more training photos with varied scenes (crowds, outdoors, several people) and augmentation that changes the scene (`mosaic`, `mixup`, `scale`). If another member's hyperparameter improves robustness to scale or scene, combining it with imgsz 800 is worth testing.

3. Challenges and how they were addressed

- **Inconsistent class IDs.** Members labelled in MakeSense with the gesture list in different orders, so Joseph's label files used different class numbers from ours. `prepare_dataset.py` checks every label file against the gesture in its file name and remaps Joseph's class IDs, leaving files that are already in our order alone. A mix-up is reported instead of silently corrupting the data.
- **Missing labels.** Amy's palm_369-376 photos had no label files at first. We held those images out until the labels were added to the shared Drive, which brought our test set to 128 photos.
- **Duplicate photos.** A perceptual-hash check found pictures that two members had both contributed. Ben replaced `dislike_380` with a new photo; the remaining pair (`dislike_388` (Daff) and `dislike_393` (Joseph)) is documented in Section 1.9.
- **Labelling errors.** Reviewing every test photo against the models' predictions found loose boxes, unlabelled hands and an unclear gesture. The group decided to keep the labels as they are and report them as human error. Every model is scored against the same labels, so comparisons between models stay fair.
- **A validation set that can't tell models apart.** Every model scores about 0.99 mAP50 on HaGRID validation. We made our own photos the headline metric and used bootstrap intervals to decide which differences are real.
- **Sharing data and compute.** We first tried Google Colab, but in the end everyone runs locally. The dataset lives in a shared Google Drive folder, and `prepare_dataset.py` builds the same seeded split from it on every machine. Training is deterministic: re-training the baseline reproduced identical metrics.
- **Noisy timings.** Training and inference times changed when the GPU was shared (the baseline took 8.5 or 15.3 min for an identical model), so speed differences of 1-2 ms, or of a few minutes of training, are not meaningful.